ПО ПРИЗНАКАМ
НОВАЯ ВЕРСИЯ

Живое объяснение 27 признаков EH/Unwind Analyzer: откуда они взялись, почему их ровно столько в версии 1.0.0, что каждый признак проверяет и как не сделать из него лишний вывод.

Этот файл написан для защиты и передачи проекта. Он не заменяет исходный код и матрицу evidence, а переводит их на нормальный человеческий язык.

Основа: `docs/evidence_matrix.md`, `src/heuristics.c`, `docs/ru/учебник по признакам/Список_27_признаков.csv`.

1. Откуда взялись 27 признаков

В проекте слово 'признак' означает не ярлык 'вредоносно', а наблюдаемый технический сигнал. Инструмент смотрит на Windows PE32+ AMD64 файл и задает очень практичный вопрос: можно ли доверять его таблицам исключений и unwind metadata, или в них есть странности, которые аналитику нужно открыть руками.

Число 27 не магическое. Это полный набор именованных findings, которые текущая версия 1.0.0 реально умеет выдавать в статической эвристической оценке. Их не выбирали ради красивого количества. Наоборот, похожие вещи объединяли, а отдельными оставляли только те сигналы, которые отличаются по смыслу, проверке или силе вывода.

• 13 признаков проверяют базовую целостность PE-секций и таблицы RUNTIME_FUNCTION. Это фундамент: если карта функций сломана, дальше анализировать опасно.

• 3 признака относятся к целостности UNWIND_INFO: читается ли оно, не выглядит ли странно, и не подтверждает ли оно проблемы размещения.

• 3 признака смотрят на массовое переиспользование unwind metadata. Такое бывает и законно, поэтому эти правила осторожные.

• 1 признак смотрит, не доминирует ли один exception handler слишком сильно.

• 7 признаков описывают размещение metadata и handler: где они лежат, какие права у секций и усиливают ли они друг друга.

Если коротко: сначала проект проверяет, не врут ли таблицы; потом - можно ли разобрать unwind metadata; потом - не лежит ли metadata в странном месте; потом - нет ли массовых повторов и централизованных handler. Последние признаки - комбинированные: они появляются, когда две отдельные странности подтверждают друг друга.

2. Как признаки отбирались

• Признак должен быть измеримым. Не 'выглядит подозрительно', а конкретно: диапазон вышел за SizeOfImage, handler попал в writable-секцию, таблица не отсортирована.

• Признак должен иметь связь с механизмом Windows x64 exception/unwind, PE-структурой или ручной проверкой в reverse engineering.

• Признак не должен раздувать ложные выводы. Поэтому есть ослабления: маленькая выборка, одинокий сигнал, только необычное имя секции без других проблем.

• Признак должен объясняться человеку. В отчете есть code, severity, family, affected count и representative RVA, чтобы аналитик мог открыть конкретное место.

• Похожие факты объединялись. Например, разные мелкие странности UNWIND_INFO сведены в unwind_anomalies, а не размножены в десятки мелких правил.

• Комбинации выделены отдельно только там, где вместе два факта реально говорят больше, чем по отдельности.

3. Как читать score и серьезность

Score в этом проекте - не процент вероятности malware. Это инженерная шкала 0..100: насколько много и насколько тяжелых сигналов набралось. Confidence - тоже не статистическая вероятность, а уверенность в интерпретации с учетом числа семейств, силы признаков и ослабляющих правил.

• Один средний признак - повод посмотреть, но не повод драматизировать.

• Высокий признак по целостности таблиц или handler обычно важнее, потому что ломает базовую модель файла.

• Несколько признаков из разных семейств говорят сильнее, чем несколько похожих признаков из одной группы.

• Маленькие файлы оцениваются осторожнее: проценты на малом количестве функций легко выглядят слишком громко.

• Комбо-признаки нужны для объяснения связки: проблема не случайно торчит в одном месте, а подтверждается рядом.

4. Быстрая карта всех признаков

N	Код признака	Семейство	Вес	Смысл простыми словами
1	`pe_section_raw_oob`	целостность PE-таблиц	+25	Секция в заголовках обещает байты, которых в файле уже нет.
2	`pe_section_virtual_oob`	целостность PE-таблиц	+20	Секция после загрузки как будто залезает за границы объявленного образа.
3	`pe_section_raw_overlap`	целостность PE-таблиц	+15	Две секции делят одни и те же байты на диске.
4	`pe_section_virtual_overlap`	целостность PE-таблиц	+20	Две секции претендуют на один и тот же участок памяти.
5	`pe_section_misaligned`	целостность PE-таблиц	+8	Размеры или адреса секций не согласованы с правилами выравнивания PE.
6	`pe_exception_directory_oob`	целостность PE-таблиц	+25	Каталог исключений, где должна лежать .pdata, указывает не туда или не читается.
7	`no_runtime_functions`	целостность PE-таблиц	+10	В x64-образе с кодом не нашлось записей RUNTIME_FUNCTION.
8	`invalid_ranges`	целостность PE-таблиц	+10 / +20	В записи функции конец стоит раньше начала или равен началу.
9	`rf_oob`	целостность PE-таблиц	+10 / +25	Функция из .pdata выходит за границы образа или понятного секционного контекста.
10	`unwind_rva_oob`	целостность PE-таблиц	+10 / +25	Запись функции указывает на UNWIND_INFO, который невозможно нормально найти в файле.
11	`rf_unsorted`	целостность PE-таблиц	+15	Таблица функций идет не по возрастанию BeginAddress.
12	`rf_overlaps`	целостность PE-таблиц	+25	Диапазоны функций пересекаются.
13	`duplicate_begin`	целостность PE-таблиц	+10	Несколько записей начинаются с одного и того же адреса функции.
14	`unwind_parse_fail`	целостность unwind-описаний	+10 / +25	UNWIND_INFO не удается разобрать до конца по правилам Windows x64.

N	Код признака	Семейство	Вес	Смысл простыми словами
15	`unwind_anomalies`	целостность unwind-описаний	+10 / +20	UNWIND_INFO читается, но внутри есть странные версии, операции, операнды, цепочки или ABI-паттерны.
16	`unwind_reuse_moderate`	массовое повторное использование unwind metadata	+10	Заметная доля функций делит один и тот же UNWIND_INFO, и рядом уже есть другой статический сигнал.
17	`unwind_reuse_strong`	массовое повторное использование unwind metadata	+20	Слишком много функций ссылается на один и тот же UNWIND_INFO.
18	`handler_dominance`	доминирование одного обработчика	+15	Один обработчик исключений обслуживает подозрительно большую часть функций.
19	`unwind_in_exec_section`	размещение metadata и handler	+20	UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код.
20	`unwind_unusual_section`	размещение metadata и handler	+8 / +15	UNWIND_INFO лежит не в привычной секции вроде .xdata, .rdata или .pdata.
21	`unwind_writable_section`	размещение metadata и handler	+10 / +18	UNWIND_INFO лежит в секции, которую можно изменять во время работы.
22	`handler_nonexec_section`	размещение metadata и handler	+15	Handler указывает туда, где код исполняться не должен.
23	`handler_writable_section`	размещение metadata и handler	+10	Handler указывает в секцию с правом записи.
24	`placement_exec_nonexec_handler_combo`	размещение metadata и handler	+10	Unwind metadata лежит как код, а handler указывает не в код.
25	`placement_integrity_combo`	целостность unwind-описаний	+8	Странное размещение metadata совпало с поломкой или странностью UNWIND_INFO.
26	`reuse_dominance_combo`	массовое повторное использование unwind metadata	+8	Массовое переиспользование unwind metadata совпало с доминированием одного handler.
27	`writable_metadata_handler_combo`	размещение metadata и handler	+10	И unwind metadata, и handler оказались связаны с writable-контекстом.

5. Подробно по каждому признаку

1. `pe_section_raw_oob`

Коротко: Секция в заголовках обещает байты, которых в файле уже нет.

Что делает: Инструмент берет таблицу секций PE и проверяет, не выходит ли область секции на диске за конец файла.

За что отвечает: Этот признак защищает весь дальнейший анализ: если секция указывает мимо файла, нельзя спокойно верить данным, которые будто бы лежат в этой секции.

Как оценивать: Это сильный сигнал. Сначала надо исключить простой вариант: файл мог быть обрезан при копировании или скачивании. Если файл целый, такой заголовок похож на намеренно испорченную или нестандартную разметку. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +25.

Осторожность: Не надо сразу говорить, что файл вредоносный. Правильнее сказать: структура файла противоречит его реальному размеру, поэтому выводы по секциям надо проверять вручную.

2. `pe_section_virtual_oob`

Коротко: Секция после загрузки как будто залезает за границы объявленного образа.

Что делает: Проверяется, укладывается ли виртуальный диапазон секции в SizeOfImage - общий размер образа, который ожидает загрузчик Windows.

За что отвечает: Признак отвечает за честность карты памяти: RVA-адреса должны попадать в понятные участки образа.

Как оценивать: Обычно это высокий риск для анализа. Если виртуальная карта сломана, один и тот же адрес может читаться неверно или вообще не иметь нормального места в образе. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +20.

Осторожность: Иногда учебные или специально сгенерированные файлы бывают кривыми. Для настоящего вывода нужно открыть PE-заголовки и проверить SizeOfImage, SectionAlignment и границы секций.

3. `pe_section_raw_overlap`

Коротко: Две секции делят одни и те же байты на диске.

Что делает: Инструмент сравнивает raw-диапазоны секций и ищет пересечения.

За что отвечает: Это ловит ситуацию, где один кусок файла можно объяснить сразу двумя секциями. Для обычной сборки это странно.

Как оценивать: Средний сигнал. Сам по себе он не приговор, но он повышает внимание: упаковщики, защитные оболочки и ручные правки иногда используют такую неоднозначность. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +15.

Осторожность: Если это единственный признак и файл маленький, не надо разгонять вывод. Нужно посмотреть, какие именно секции пересекаются и используются ли они реально.

4. pe_section_virtual_overlap

Коротко: Две секции претендуют на один и тот же участок памяти.

Что делает: Проверяется пересечение виртуальных адресов секций после загрузки.

За что отвечает: Признак отвечает за однозначность адресного пространства PE: один RVA должен вести в один понятный участок.

Как оценивать: Это сильнее, чем raw-overlap. Если память размечена неоднозначно, анализ handler и unwind metadata может уехать в неправильное место. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +20.

Осторожность: Нужно сверить, как такой файл реально принимает загрузчик Windows. Но для статического анализа это уже красная зона.

5. pe_section_misaligned

Коротко: Размеры или адреса секций не согласованы с правилами выравнивания PE.

Что делает: Инструмент проверяет FileAlignment, SectionAlignment и связанные инварианты заголовков.

За что отвечает: Выравнивание - это бытовая дисциплина формата. Когда она нарушена, парсер уже не может читать образ как обычный compiler output.

Как оценивать: Средний сигнал. Он полезен как фон: один misalignment может быть особенностью генератора, но вместе с другими признаками становится важным подтверждением. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +8.

Осторожность: Сначала смотрят масштаб: одно небольшое отклонение и пачка сломанных секций - это разные истории.

6. pe_exception_directory_oob

Коротко: Каталог исключений, где должна лежать .pdata, указывает не туда или не читается.

Что делает: Проверяется Exception Directory из PE Data Directory: его RVA и размер должны переводиться в реальные байты.

За что отвечает: Это входная дверь всего EH/unwind анализа. Если она сломана, таблицу функций нельзя считать надежной.

Как оценивать: Высокий сигнал. Для Windows x64 корректная exception directory важна для нормального unwind поведения. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +25.

Осторожность: Если каталог отсутствует у файла без кода, это может быть нормально. Но если есть исполняемые секции, а exception directory сломана, это надо показывать как серьезный факт.

7. no_runtime_functions

Коротко: В x64-образе с кодом не нашлось записей RUNTIME_FUNCTION.

Что делает: Инструмент видит, что exception directory пустая, а исполняемый код в образе есть.

За что отвечает: Записи RUNTIME_FUNCTION помогают Windows понимать границы функций и раскручивать стек при исключениях.

Как оценивать: Это средний и осторожный признак. Маленький leaf-only код теоретически может обходиться без таких записей. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +10.

Осторожность: Нельзя оценивать его грубо. Надо посмотреть, есть ли сложные функции, вызовы, прологи, stack frame. Если код реально нетривиальный, отсутствие .pdata выглядит подозрительнее.

8. `invalid_ranges`

Коротко: В записи функции конец стоит раньше начала или равен началу.

Что делает: Для каждой RUNTIME_FUNCTION проверяется диапазон [BeginAddress, EndAddress).

За что отвечает: Признак отвечает за базовую геометрию функции: функция должна занимать ненулевой и направленный диапазон адресов.

Как оценивать: Если таких записей мало, это средний сигнал. Если больше 1 процента таблицы - уже высокий: это не похоже на случайную мелочь. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium` if `<=1%`, else `high, вклад в score - +10 / +20.

Осторожность: Нужно смотреть affected count и процент. Одна битая запись в огромном файле и системная поломка таблицы оцениваются по-разному.

9. `rf_oob`

Коротко: Функция из .pdata выходит за границы образа или понятного секционного контекста.

Что делает: BeginAddress и EndAddress сверяются с SizeOfImage и секциями.

За что отвечает: Этот признак ловит RUNTIME_FUNCTION, которая описывает код там, где кода в нормальной карте образа нет.

Как оценивать: До 1 процента - средний сигнал, выше - высокий. Чем больше таких записей, тем меньше доверия к таблице. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium` if `<=1%`, else `high, вклад в score - +10 / +25.

Осторожность: Важно проверить, не анализируется ли намеренно поврежденный тестовый образ. В реальном ПО такие диапазоны требуют ручной проверки.

10. `unwind_rva_oob`

Коротко: Запись функции указывает на UNWIND_INFO, который невозможно нормально найти в файле.

Что делает: UnwindInfoRVA переводится в file offset и секцию; если адрес не мапится, признак срабатывает.

За что отвечает: RUNTIME_FUNCTION без читаемого UNWIND_INFO теряет смысл: есть функция, но нет нормальной инструкции, как раскручивать стек.

Как оценивать: Порог такой же: мало - средний сигнал, системно - высокий. Особенно важно, если вместе есть parse fail или странное размещение metadata. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium` if `<=1%`, else `high, вклад в score - +10 / +25.

Осторожность: Не путать с отсутствием handler. Handler может не быть, а UNWIND_INFO все равно должен читаться.

11. `rf_unsorted`

Коротко: Таблица функций идет не по возрастанию BeginAddress.

Что делает: Инструмент проходит RUNTIME_FUNCTION и считает инверсии порядка.

За что отвечает: Windows и инструменты ожидают предсказуемую таблицу. Несортированность мешает быстрому и однозначному поиску функции по адресу.

Как оценивать: Средний сигнал. Он показывает не столько поломку одной функции, сколько грязную или искусственную организацию таблицы. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +15.

Осторожность: Если есть только этот признак, лучше говорить осторожно: таблица нестандартно упорядочена, нужна сверка с остальными симптомами.

12. `rf_overlaps`

Коротко: Диапазоны функций пересекаются.

Что делает: После сортировки инструмент смотрит, не залезает ли одна RUNTIME_FUNCTION в адреса другой.

За что отвечает: Один адрес кода не должен одновременно принадлежать двум нормальным функциям в этой таблице.

Как оценивать: Высокий сигнал, потому что пересечение ломает саму идею надежной карты функций. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - high, вклад в score - +25.

Осторожность: Стоит открыть конкретные RVA и посмотреть, это ручная правка, защита, генератор кода или повреждение файла.

13. `duplicate_begin`

Коротко: Несколько записей начинаются с одного и того же адреса функции.

Что делает: Инструмент ищет одинаковые BeginAddress в RUNTIME_FUNCTION.

За что отвечает: Признак ловит дубли и неоднозначность: один старт функции получает несколько разных описаний.

Как оценивать: Средний сигнал. Он становится сильнее, если дубли ведут к разным unwind info или идут вместе с overlap. В матрице проекта: семейство - целостность PE-таблиц, правило серьезности - medium, вклад в score - +10.

Осторожность: Одинаковое начало не всегда объясняет всю картину. Нужно смотреть, чем отличаются записи и куда они указывают.

14. `unwind_parse_fail`

Коротко: UNWIND_INFO не удается разобрать до конца по правилам Windows x64.

Что делает: Парсер читает version, flags, opcodes, handler/chained tail и останавливается, если структура физически не сходится.

За что отвечает: Это один из главных признаков целостности: metadata есть, но ее нельзя прочитать как нормальное unwind-описание.

Как оценивать: До 1 процента - средний сигнал, больше - высокий. Массовые parse fail обычно говорят не о случайности, а о системной проблеме. В матрице проекта: семейство - целостность unwind-описаний, правило серьезности - medium` if `<=1%`, else `high, вклад в score - +10 / +25.

Осторожность: Формальные ABI-предупреждения в strict mode не равны parse fail. Здесь речь именно о том, что структура не читается корректно.

15. `unwind_anomalies`

Коротко: UNWIND_INFO читается, но внутри есть странные версии, операции, операнды, цепочки или ABI-паттерны.

Что делает: Инструмент не падает на записи, а собирает необычные свойства: opcode, opinfo, chain, epilog и другие детали.

За что отвечает: Этот признак нужен для серой зоны: файл не сломан полностью, но его unwind metadata выглядит нетипично.

Как оценивать: Если затронуто до 5 процентов функций - средний сигнал. Если больше - высокий, потому что странность уже стала массовым шаблоном. В матрице проекта: семейство - целостность unwind-описаний, правило серьезности - medium` if `<=5%`, else `high, вклад в score - +10 / +20.

Осторожность: Нужно различать 'нестандартно' и 'невозможно'. Некоторые компиляторы дают необычные, но допустимые варианты.

16. `unwind_reuse_moderate`

Коротко: Заметная доля функций делит один и тот же UNWIND_INFO, и рядом уже есть другой статический сигнал.

Что делает: Инструмент считает, сколько функций ссылаются на каждый UnwindInfoRVA; умеренный reuse включается только при дополнительном подтверждении.

За что отвечает: Признак отвечает за поиск централизованной metadata-схемы, где много функций ведут в одну точку описания.

Как оценивать: Это средний сигнал и специально осторожный: он не срабатывает просто от reuse, если больше ничего странного нет. В матрице проекта: семейство - массовое повторное использование unwind metadata, правило серьезности - medium, вклад в score - +10.

Осторожность: Нормальные компиляторы тоже могут переиспользовать одинаковые unwind-описания. Поэтому здесь обязательно нужен соседний признак.

17. unwind_reuse_strong

Коротко: Слишком много функций ссылается на один и тот же UNWIND_INFO.

Что делает: Если в большом образе больше 30 процентов функций делят один UnwindInfoRVA, инструмент выдает сильный reuse-признак.

За что отвечает: Это ловит грубую централизацию unwind metadata, характерную для оберток, диспетчеров или искусственно построенных таблиц.

Как оценивать: Высокий сигнал, но только на достаточно большой таблице. В маленьком файле проценты легко обманывают. В матрице проекта: семейство - массовое повторное использование unwind metadata, правило серьезности - high, вклад в score - +20.

Осторожность: Нужно смотреть размер корпуса функций. На пяти функциях '40 процентов' может означать всего две записи.

18. handler_dominance

Коротко: Один обработчик исключений обслуживает подозрительно большую часть функций.

Что делает: Инструмент считает HandlerRVA и смотрит, не доминирует ли один handler среди большого числа записей.

За что отвечает: Обычная программа может иметь общие runtime handlers, но тотальное доминирование похоже на центральный диспетчер.

Как оценивать: Средний сигнал. В реализации он смягчен: если handler выглядит как нормальный runtime-код и нет других статических проблем, признак не раздувается. В матрице проекта: семейство - доминирование одного обработчика, правило серьезности - medium, вклад в score - +15.

Осторожность: На защите важно сказать честно: это не доказательство защиты или упаковщика, а повод проверить, почему один handler так часто повторяется.

19. unwind_in_exec_section

Коротко: UNWIND_INFO лежит в исполняемой секции, где обычно ожидается код.

Что делает: Адрес unwind metadata связывается с секцией; если у секции стоит право execute, признак срабатывает.

За что отвечает: Metadata обычно живет в .xdata, .rdata, .pdata или похожих data-секциях. Исполняемая секция для нее выглядит странно.

Как оценивать: Высокий сигнал. Такое размещение труднее объяснить обычной компиляцией. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - high, вклад в score - +20.

Осторожность: Нужно посмотреть имя секции и права. Если это специально названная, но executable-секция, право execute все равно остается важным фактом.

20. unwind_unusual_section

Коротко: UNWIND_INFO лежит не в привычной секции вроде .xdata, .rdata или .pdata.

Что делает: Инструмент проверяет имя и контекст секции, где находится unwind metadata.

За что отвечает: Признак отвечает за бытовую нормальность размещения: metadata должна лежать там, где ее ожидают инструменты и загрузчик.

Как оценивать: До 5 процентов - средний сигнал, больше - высокий. Массовое необычное размещение говорит сильнее, чем одна запись. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - medium` if `<=5%`, else `high, вклад в score - +8 / +15.

Осторожность: Этот признак специально может ослабляться, если он один. Странное имя секции еще не равно повреждению.

21. unwind_writable_section

Коротко: UNWIND_INFO лежит в секции, которую можно изменять во время работы.

Что делает: Проверяется флаг write у секции, где лежит unwind metadata.

За что отвечает: Unwind metadata обычно воспринимается как описание, а не как изменяемые рабочие данные. Writable-контекст повышает риск подмены.

Как оценивать: Средний или высокий сигнал: больше 5 процентов затронутых записей - уже широкий шаблон. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - medium` if `<=5%`, else `high, вклад в score - +10 / +18.

Осторожность: Некоторые runtime-сценарии могут строить таблицы динамически, но для статического файла writable metadata - веская причина открыть объект руками.

22. handler_nonexec_section

Коротко: Handler указывает туда, где код исполняться не должен.

Что делает: HandlerRVA связывается с секцией; если секция не executable или вообще не находится, признак срабатывает.

За что отвечает: Обработчик исключения - это код. Если адрес handler ведет в неисполняемые данные, что-то в модели файла не сходится.

Как оценивать: Высокий сигнал. Это один из самых понятных для объяснения признаков размещения. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - high, вклад в score - +15.

Осторожность: Стоит проверить, не ошибка ли это парсинга из-за уже сломанной UNWIND_INFO. Если parse fail рядом есть, признаки усиливают друг друга.

23. handler_writable_section

Коротко: Handler указывает в секцию с правом записи.

Что делает: Инструмент проверяет write-флаг у секции, куда попадает HandlerRVA.

За что отвечает: Код handler в writable-секции выглядит менее надежно: теоретически его можно менять как данные.

Как оценивать: Средний сигнал. Сам по себе он не такой сильный, как non-exec handler, но вместе с writable unwind metadata становится важнее. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - medium, вклад в score - +10.

Осторожность: Нужно смотреть сочетания прав секции: execute+write и просто write без execute - разные случаи.

24. `placement_exec_nonexec_handler_combo`

Коротко: Unwind metadata лежит как код, а handler указывает не в код.

Что делает: Это комбинированный признак: он появляется, когда одновременно есть unwind_in_exec_section и handler_nonexec_section.

За что отвечает: Комбинация показывает, что странность не одиночная: обе стороны exception-механизма размещены нелогично.

Как оценивать: Высокий сигнал. Его стоит объяснять как усиление: два связанных факта вместе говорят больше, чем каждый по отдельности. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - high, вклад в score - +10.

Осторожность: Комбо не заменяет ручной анализ. Оно говорит: 'смотри на пару unwind metadata + handler как на один подозрительный узел'.

25. `placement_integrity_combo`

Коротко: Странное размещение metadata совпало с поломкой или странностью UNWIND_INFO.

Что делает: Признак появляется, когда placement-аномалия идет вместе с parse fail, unwind anomalies или недоступным UnwindInfoRVA.

За что отвечает: Он отделяет просто необычное место от ситуации, где необычное место еще и плохо читается.

Как оценивать: Средний усилитель. Он не самый тяжелый по весу, но хорошо повышает доверие к тому, что проблема настоящая. В матрице проекта: семейство - целостность unwind-описаний, правило серьезности - medium, вклад в score - +8.

Осторожность: Важно называть его именно подтверждающим признаком. Он не первичный факт, а связка двух фактов.

26. `reuse_dominance_combo`

Коротко: Массовое переиспользование unwind metadata совпало с доминированием одного handler.

Что делает: Инструмент связывает две статистики: много функций делят один UnwindInfoRVA и много функций идут к одному HandlerRVA.

За что отвечает: Такой рисунок похож на центральный диспетчер: много функций формально разные, но exception-механизм у них сведен в одну схему.

Как оценивать: Средний усилитель. Он ценен не весом сам по себе, а тем, что соединяет reuse и handler behavior. В матрице проекта: семейство - массовое повторное использование unwind metadata, правило серьезности - medium, вклад в score - +8.

Осторожность: Нужно проверять на крупных файлах. На маленьких выборках доли могут выглядеть громче, чем есть на самом деле.

27. `writable_metadata_handler_combo`

Коротко: И unwind metadata, и handler оказались связаны с writable-контекстом.

Что делает: Комбо срабатывает, когда одновременно есть writable unwind metadata и writable handler target.

За что отвечает: Обе части механизма исключений оказываются изменяемыми. Для обычного статического образа это трудно назвать спокойной картиной.

Как оценивать: Высокий сигнал. Его удобно объяснять как связанное подтверждение: меняемой выглядит не одна случайная деталь, а вся линия metadata -> handler. В матрице проекта: семейство - размещение metadata и handler, правило серьезности - high, вклад в score - +10.

Осторожность: Ручная проверка обязательна: нужно посмотреть права секций, конкретные RVA и то, является ли файл обычным PE или специальным runtime-генератором.

6. Что говорить на защите

Самая честная формулировка такая: проект не пытается 'угадать вредоносность'. Он строит проверяемую карту EH/unwind metadata и отмечает места, где эта карта противоречит нормальной структуре Windows PE32+ AMD64 или выглядит нетипично для обычной компиляции.

27 признаков - это не случайная цифра и не маркетинговый список. Это набор устойчивых, реализованных и документированных сигналов версии 1.0.0. Они покрывают пять смысловых зон: целостность таблиц, целостность UNWIND_INFO, размещение metadata, повторное использование metadata и доминирование обработчика.

Правильный вывод по отчету звучит не 'score 70 значит 70 процентов опасности', а 'в файле набралось много технических сигналов, часть из них сильные, они затрагивают такие-то семейства, поэтому требуется ручная проверка конкретных RVA и секций'.

Именно поэтому в проекте рядом со score есть severity, confidence, affected count, representative RVA, mitigations и manual validation steps. Машина не заменяет аналитика, а быстро подсвечивает места, куда аналитику стоит смотреть первым.

7. Разбиение 27 признаков по шести слоям анализа

Этот раздел добавляет к списку признаков еще одну координатную систему. Важно: шесть слоев - это не новые findings и не замена пяти evidence-семейств из матрицы. Evidence-семейства отвечают за score, severity и confidence. Шесть слоев отвечают за объяснение: на каком уровне анализа обнаружен сигнал и что именно нужно проверить руками.

В текущей версии проекта 27 structured findings являются прежде всего статическими признаками PE32+ AMD64 и EH/unwind metadata. Поэтому основная часть признаков ложится на слои 2-5. Слой 1 появляется, когда есть конкретная машинная точка выполнения или падения. Слой 6 появляется при динамической или смешанной проверке, когда результат virtual unwind сравнивается с нормальной цепочкой кадров.

Слой	Что проверяется	За что отвечает в проекте
1. Машинная точка	RIP, текущая инструкция, регистры, тип доступа к памяти, непосредственная причина сбоя или перехода.	Показывает, что произошло в конкретной точке исполнения. Для 27 статических findings это контекстный слой: он нужен при crash-анализе, отладке или динамическом тесте.
2. PE-образ	ImageBase, RVA, принадлежность адресу модулю, границы и права секций, таблицы директорий PE.	Отвечает за координатную систему. Здесь проверяется, не указывают ли структуры файла за пределы образа и не лежат ли данные в странном месте.
3. Function table	.pdata и RUNTIME_FUNCTION: BeginAddress, EndAddress, UnwindInfoAddress, порядок, пересечения, покрытие функций.	Отвечает за карту функций. Если карта повреждена, virtual unwind может выбрать неверную функцию или неверный xdata-блок.
4. Unwind info	UNWIND_INFO: version, flags, prolog size, unwind codes, frame register, chained info, корректность разбора.	Отвечает за рецепт выхода из stack frame: можно ли восстановить регистры/стек и перейти к вызывающему кадру.
5. Handler layer	UNW_FLAG_EHANDLER, UNW_FLAG_UHANDLER, handler RVA, секция handler-а, language-specific данные.	Отвечает за логику обработки исключений: куда передается управление и нормально ли выглядит handler target.
6. Stack walking result	Итог virtual unwind: цепочка кадров, обрыв, повторяющиеся кадры, адреса вне модулей, возвраты в неисполняемые области.	Отвечает за финальную историю выполнения. Для текущих статических findings это слой валидации, показывающий последствия проблем слоев 2-5.

Схема прохождения анализа

`1. RIP/instruction` -> `2. PE image/RVA` -> `3. RUNTIME_FUNCTION` -> `4. UNWIND_INFO` -> `5. Handler` -> `6. Stack walk result`

На защите это можно объяснять так: сначала определяется машинная точка, потом адрес переводится в координаты PE-образа, затем ищется запись функции, после этого читается unwind-рецепт, проверяется обработчик исключения, а в конце оценивается цепочка кадров, которую дает virtual unwind.

Привязка всех 27 findings к слоям

В таблице указан основной слой, где рождается смысл признака. Связанные слои нужны для пограничных случаев: например, handler относится к слою 5, но его подозрительность часто доказывается через права секции со слоя 2.

Finding code	Основной слой	Что означает привязка	Связанные
`pe_section_raw_oob`	2. PE-образ	Raw-диапазон секции выходит за границы файла.	2
`pe_section_virtual_oob`	2. PE-образ	Виртуальный диапазон секции выходит за границы image layout.	2
`pe_section_raw_overlap`	2. PE-образ	Raw-диапазоны секций пересекаются, байты файла получают неоднозначную принадлежность.	2
`pe_section_virtual_overlap`	2. PE-образ	Виртуальные диапазоны секций пересекаются, один RVA может попадать в несколько секций.	2
`pe_section_misaligned`	2. PE-образ	Нарушено ожидаемое выравнивание секций относительно FileAlignment/SectionAlignment.	2
`pe_exception_directory_oob`	2. PE-образ	Exception Directory как PE data directory указывает за пределы корректного образа.	2, 3
`no_runtime_functions`	3. Function table	Отсутствуют записи RUNTIME_FUNCTION там, где анализ ожидает карту функций.	3

Finding code	Основной слой	Что означает привязка	Связанные
invalid_ranges	3. Function table	BeginAddress/EndAddress образуют нулевой, обратный или бессмысленный диапазон функции.	3
rf_oob	3. Function table	Диапазоны RUNTIME_FUNCTION выводят функцию за пределы допустимого PE-образа.	3, 2
unwind_rva_oob	3. Function table	Поле UnwindInfoAddress в RUNTIME_FUNCTION ведет в недопустимый RVA.	3, 4, 2
rf_unsorted	3. Function table	Function table не отсортирована, из-за чего поиск функции по RIP/RVA становится ненадежным.	3
rf_overlaps	3. Function table	Диапазоны функций перекрываются, один адрес может принадлежать нескольким RUNTIME_FUNCTION.	3
duplicate_begin	3. Function table	Несколько записей имеют одинаковый BeginAddress, начало функции становится неоднозначным.	3
unwind_parse_fail	4. Unwind info	UNWIND_INFO невозможно корректно разобрать, рецепт раскрутки кадра не получается.	4, 3
unwind_anomalies	4. Unwind info	В UNWIND_INFO есть структурные странности: поля, flags/codes или chained layout выглядят некорректно.	4
unwind_reuse_moderate	4. Unwind info	Один UnwindInfoRVA умеренно часто используется многими функциями, сигнал важен вместе с подтверждением.	4, 3
unwind_reuse_strong	4. Unwind info	Сильная концентрация функций на одном UnwindInfoRVA может указывать на искусственную metadata.	4, 3
handler_dominance	5. Handler layer	Один handler target доминирует над большим числом функций.	5, 4
unwind_in_exec_section	4. Unwind info	UNWIND_INFO лежит в исполняемой секции, хотя metadata обычно не должна быть исполняемой.	4, 2
unwind_unusual_section	4. Unwind info	UNWIND_INFO лежит в нетипичной секции, что проверяется через карту PE-секций.	4, 2
unwind_writable_section	4. Unwind info	UNWIND_INFO лежит в writable-секции, рецепт раскрутки кадра потенциально изменяем.	4, 2
handler_nonexec_section	5. Handler layer	Handler target указывает в неисполняемую секцию, что странно для передачи управления.	5, 2
handler_writable_section	5. Handler layer	Handler target связан с writable-секцией, логика обработки оказывается в изменяемом контексте.	5, 2
placement_exec_nonexec_handler_combo	5. Handler layer	Комбинация подозрительного размещения unwind metadata и handler target.	5, 4, 2
placement_integrity_combo	4. Unwind info	Странное размещение metadata подтверждается структурным повреждением unwind-информации.	4, 2, 3
reuse_dominance_combo	5. Handler layer	Повторное использование unwind metadata связано с доминированием одного handler-а.	5, 4
writable_metadata_handler_combo	5. Handler layer	Writable выглядит и unwind metadata, и handler target, поэтому вся EH/unwind-линия подозрительна.	5, 4, 2

Контрольный список 27 кодов

pe_section_raw_oob, pe_section_virtual_oob, pe_section_raw_overlap, pe_section_virtual_overlap, pe_section_misaligned, pe_exception_directory_oob, no_runtime_functions, invalid_ranges, rf_oob, unwind_rva_oob, rf_unsorted, rf_overlaps, duplicate_begin, unwind_parse_fail

unwind_anomalies, unwind_reuse_moderate, unwind_reuse_strong, handler_dominance, unwind_in_exec_section, unwind_unusual_section, unwind_writable_section, handler_nonexec_section, handler_writable_section, placement_exec_nonexec_handler_combo, placement_integrity_combo, reuse_dominance_combo, writable_metadata_handler_combo

Как читать это разбиение на защите

Главная мысль: findings проекта не обязаны равномерно покрывать все шесть слоев. Текущая версия анализатора статическая, поэтому она подробно проверяет PE-образ, function table, UNWIND_INFO и handler layer. Машинная точка и итоговый stack walk нужны для динамической интерпретации и ручной валидации.

Если спрашивают, почему слой 1 почти не представлен в таблице findings: слой 1 требует конкретного RIP и состояния CPU. Статический анализатор работает с файлом, а не с исполняющимся процессом. Он готовит координаты и признаки, которые затем можно сопоставить с RIP при отладке или crash-анализе.

Если спрашивают про слой 6: слой 6 показывает последствие. Когда слои 2-5 повреждены, virtual unwind может оборваться, зациклиться, уйти в адрес вне модуля или дать переход в неисполняемую область. В документе 27 признаков описывают причины, а слой 6 описывает ожидаемый эффект при динамической проверке.

Практическая формула: назвать finding code, назвать слой, сказать, какую структуру он проверяет, показать representative RVA/секцию из отчета, и только после этого делать осторожный вывод о необходимости ручной проверки.

Короткая защита формулировки

> Корректно говорить так: “27 findings - это реализованные статические признаки. Шесть слоев - это модель объяснения анализа от конкретной машинной точки до результата раскрутки стека. В моей версии основная проверка находится на слоях PE image, function table, unwind info и handler layer, a machine point и stack walking result используются при динамической и смешанной валидации”.

Такое объяснение не противоречит матрице evidence. Матрица отвечает за веса, severity и confidence. Слои отвечают за архитектурную интерпретацию признаков и помогают показать, почему анализатор проверяет именно эти структуры Windows x64 EH/unwind.